Imitation Learning of a Humanoid Burpee Motion in MuJoCo via Curriculum-Guided PPO

And we thought, this would be easy....

Cover Sheet

Team members: Sven Neugebauer [SN], Louisa Wessel [LW]

Git repository: https://github.com/SvnNgbr/nca-sn-lw-mujoco

Presentation: https://www.figma.com/board/lRynw06d1QNUb4VMZNYzNT/NC-Assignment...

Individual contributions

Section / Task	Contributor(s)	Notes
Environment design (BurpeeImitationEnvV2)	*LW*	[TODO: TODO]
Reward shaping	*SN, LW*	groundwork by LW, than adapted by SN for v5
Reference motion (burpee_reference.py)	*LW*	[TODO: TODO]
Curriculum design & training	*SN, LW*	switch to phasechange by SN, xxx LW
Baseline PPO training	*SN, LW*	
Adaptation of BurpeeImitationEnv V2 to Humanoid-V5	*SN*	Includes adapting env to v5env.py, new v5ref.py, creation of DEBUG functions and new v5train.py
Plot / video generation scripts	*SN, LW*	Script created with phasechange and code for plotting by SN, finetuned to new XML by LW
Report basic structure	*SN*	general structure outline and small helping hints for later
Background: repo setup	*SN*	cleanup and management of branches. Also automated installation script and readme file.
Background: centralised run	*SN*	run.py fine that can be used to automatically run all scripts (only one command to run instead of 3)
After here no coding and only report work		
Abstract	*SN*	
Introduction	*SN, LW*	1.6 by LW, rest SN
Methodology	*SN, LW*	2.6 by LW, rest SN
Results	*SN, LW*	3.5 by SN & LW, rest SN
Discussion	*SN*	

Abstract / Summary

This work extends an existing approach to motion imitation for a humanoid roboter. The focus is on transferring a burpee reference trajectory tracking system (movement) from an idea, to a first (prototype) self-defined MuJoCo model to the standardised Humanoid_v5 from the Gymnasium environment. In our case we copied the XML from Gymnasium and did not use the import function.

The main challenge was adopting the joint structure and the reward function to the different models. The introduction of a hold reward was intended to improve stability during the ground-based phases of the burpee.

The results show that the agent is able to imitate the standing phases of the movement well. The dynamic and ground based phases, in particular the transition into the plank and the push-up movement, remain a challenge, which would need further work. We will discuss this more at the end.

1. Introduction

1.1 Big picture challenge

From a neuromorphic standpoint, the control of a humanoid robot is of interest, because it involves translating discrete motor commands into continuous, stable movement.

A burpee is particularly complex as it comprises several phases: standing to squat, transition to ground contact, plank, push-up and then the process of getting on your feet again followed by a jump. Each phase places different demands on balance, strength and coordination.

This project utilises reinforcement learning (PPO) with the reference trajectory to guide exploration within this high-dimensional movement space.

1.2 Prior approaches

[TODO: Classical trajectory tracking / PD control vs. RL-based imitation (e.g. DeepMimic-style methods), physics simulators like MuJoCo. Cite 2-4 sources.] maybe auch nix weiter sonder so stehen lassen...

ExBody2 (Advanced Expressive Humanoid Whole-Body Control)

ZEST (Zero-shot Embodied Skill Transfer)

SMILE (Single Model Imitation Learning for Multi-Skill Efficiency)

HumanMimic (Learning Natural Locomotion and Transitions via Wasserstein Adversarial Imitation)

1.3 Why that's not enough

[TODO: e.g. naive RL-from-scratch has hard exploration on dynamic, ground-contact-heavy motions like a burpee; pure trajectory tracking is brittle without a learned residual/balance component]

Existing frameworks such as ExBody2, ZEST, SMILE, and HumanMimic demonstrate impressive whole-body imitation and skill transfer. However, they rely on large motion capture datasets, complex architectures, and substantial computational resources. We were not aware of these prior works when we started, and instead developed our own task-specific implementation for the burpee motion. Later while writing this report we came across those projects. While such frameworks are powerful for general-purpose imitation, our custom approach is sufficient for this structured, keyframe-defined motion.

1.4 Our approach & contribution

The main focus lies in creating a successful porting from the classical burpee in a human to our first model (burpee imitation) into a learning framework with the Humanoid_v5. This involved a complete redefinition of the joint names and reference trajectory. The reward function was extended to include a hold reward, which rewards the agent for maintaining a correct pose, which therefore results in him not learning to collapse in the first steps. This report and the git-repo documents the challenges and adaptations encountered when porting the framework to the new model.

1.5 Specific problem statement

Train a simulated MuJoCo humanoid to perform a complete burpee sequence robustly, starting from a reference trajectory and residual RL control. the agent should mimic a given reference as closely as possible, whilst developing a certain degree of intrinsic stability. In the end the training should rely on a robust PPO algorithm and a curriculum that gradually reduces the root-assist.

1.6 Connection to the lecture

[TODO: Explicitly tie to the lecture concepts you're meant to demonstrate — e.g. reference tracking / central pattern generators / reward shaping / sim-to-real, whichever your course covered]

2. Methodology

2.1 Simulator & base assets

- MuJoCo physics engine, humanoid model: humanoid_burpee.xml
- Humanoid_v5.xml
- Base MuJoCo repository forked/extended (see setup.py — clones and patches the upstream repo)

2.2 Reference motion

- burpee_reference.py: defines JOINT_NAMES, TOTAL_DURATION, reference_at(t) returning target joints/root pose/phase
- The reference trajectory for the v5 humanoid has been defined in the file v5ref.py. (It is based on the same keyframes as the v3 model, but with angles adapted to the v5 joint structure.)
- The joint names were determined by querying the MuJoCo model parameters. The order of the joints is: abdomen_z, abdomen_y, abdomen_x, right_hip_x, right_hip_z, right_hip_y, right_knee, left_hip_x, left_hip_z, left_hip_y, left_knee, right_shoulder1, right_shoulder2, right_elbow, left_shoulder1, left_shoulder2, left_elbow.
- The pose was validated iteratively using an interactive pose tester (v5posmove.py).

2.3 Environment

2.3.1 (BurpeeImitationEnvV2)

- Observation space: qpos, qvel, joint tracking error, root position/orientation error, reference joints, phase signal
- Action space: residual action added to reference control (action_scale), clipped to actuator range
- Root-assist mechanism: blends simulated root pose toward reference by root_assist factor
- Episode termination: root height bounds, episode duration (TOTAL_DURATION)

2.3.2 (Humanoid_v5)

- The environment has been adapted for the Humanoid v5. The most significant changes relate to the joint-mapping logic and the observation space dimension.
- The root-assist mechanism has been carried over from previous work. It interpolates the root position of the simulated robot with the reference root position. The root_assist factor is reduced during the curriculum.
- During the floor phases (plank, push-up), the episode termination threshold has been relaxed (root_height < 0.12 instead of 0.3) to allow the robot to assume the correct low pose.

2.4 Reward design

Weighted sum of:

- Pose tracking reward (joint error)
- Root position reward
- Root orientation reward

- Control tracking reward
- Action smoothness reward
- Alive bonus / fall penalty
- Hold-Bonus
- Alive-Bonus
- Hight reward
- Control Cost

Rewards in BurpeeImitationV2:

```
pose_reward = np.exp(-8.0 * np.mean(joint_error * joint_error))
root_reward = np.exp(-10.0 * np.mean(root_pos_error * root_pos_error))
orient_reward = np.exp(-4.0 * np.mean(root_quat_error * root_quat_error))
control_reward = np.exp(-0.5 * np.mean(ctrl_error * ctrl_error))
smooth_reward = np.exp(-0.1 * np.mean(action_delta * action_delta))
alive_bonus = 0.2 if root_height > 0.12 else -1.0

return (2.5 * pose_reward +
        1.5 * root_reward +
        0.8 * orient_reward +
        0.6 * control_reward +
        0.3 * smooth_reward +
        alive_bonus)
```

Rewards in Humanoid_V5

```
pose_reward = np.exp(-5.0 * np.mean(joint_error * joint_error))
height_reward = np.exp(-3.0 * (root_height - target_height)**2)
root_reward = np.exp(-2.0 * np.mean(root_pos_error[:2]**2))

hold_bonus = 0.0
if self.pose_held:
    hold_bonus = 2.0
elif self.phase_hold_time > 0:
    hold_bonus = 0.5 * min(self.phase_hold_time / self.hold_seconds, 1.0)

alive_bonus = 0.5 if root_height > 0.15 else -1.0
control_cost = -0.01 * np.mean(ctrl * ctrl)

return (2.0 * pose_reward +
        1.0 * height_reward +
        0.5 * root_reward +
        hold_bonus +
        alive_bonus +
        control_cost)
```

Why did we change them?

- The original reward function contained many exponential terms, some of which were redundant. The current version is more streamlined.
- Instead of rewarding root orientation, the correct body height is now rewarded. This is more intuitive for the ground phases of the burpee.
- A hold bonus to encourage the robot to hold a pose rather than simply moving into it briefly.
- Instead of rewarding control magnitude and smoothness, large movements are now penalised directly. This is more efficient.

2.5 Learning algorithm

- The curriculum gradually reduces the root assist from 0.98 via 0.75 via 0.45 and 0.15 to 0.0.
- The high root assist at the start of training allows the agent to learn the basic movement sequence without having to worry about balance.
- As the assistance decreases, the agent must increasingly maintain its own stability. This promotes the development of a robust policy.
- The hold time (hold_seconds) was set to 0.5 seconds during training to speed up the learning process. For the final evaluation, it was increased to 2.0 seconds.
- [TODO: PPO — library used (e.g. Stable-Baselines3), version]
- [TODO: Key hyperparameters — learning rate, batch size, n_steps, total timesteps, etc.]

2.6 Curriculum strategy

[TODO: What varies across the curriculum runs — e.g. root_assist annealed from high to low, difficulty stages — and why this should help vs. flat PPO]

2.7 What we built vs. what we reused

Component	Reused	Our contribution
MuJoCo engine	Yes (external)	—
Base MuJoCo repo (forked)	Yes (external base)	*[TODO: TODO]*
PPO implementation	Yes (external library)	Hyperparameter tuning
BurpeeImitationEnvV2	No	Built by us
Humanoid-V5.xml	yes	-
numerous V5xxx.py	no	Built by us
Reward shaping	No	Built by us
Reference motion	no	Built by us
Curriculum scheduler	No	Built by us
Debug functions	no	Built by us

3. Results

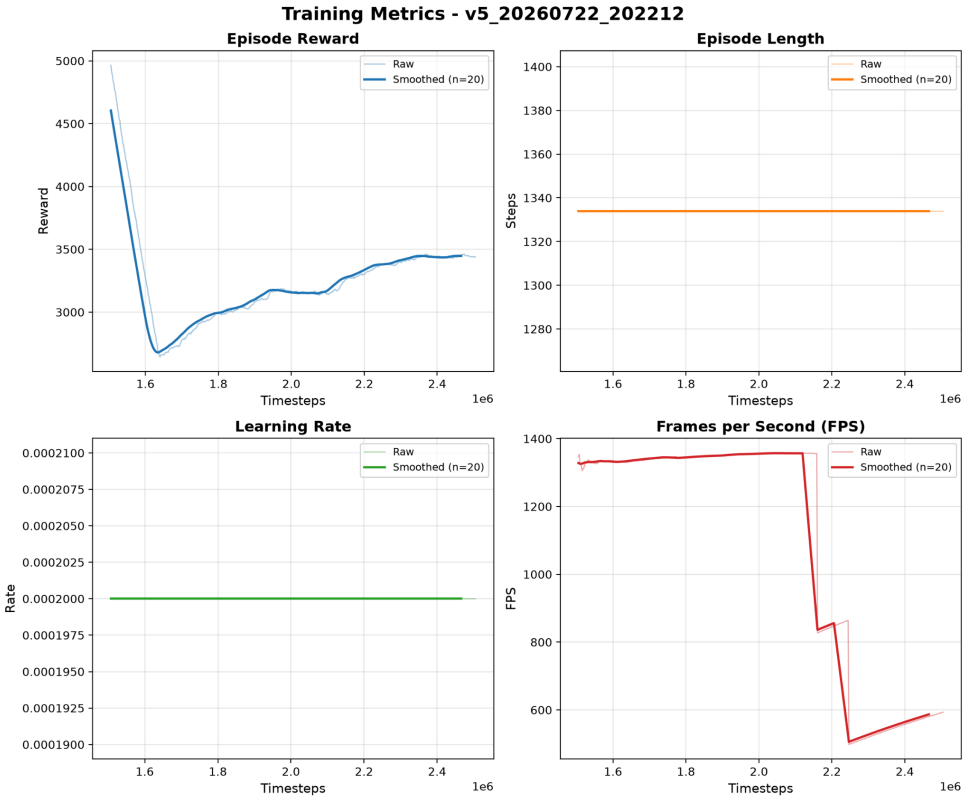
3.1 Experiments run

- Baseline PPO (no curriculum): runs/PPO_1
- Curriculum PPO: runs_curriculum/PPO_0
- Iteration history across design changes: Code_Iterationen/ (removed in the final project for convenience) (see repo history)
- training with the v3 model (as a baseline) and training with the Humanoid v5.
- For the Humanoid v5, several iterations with different reward parameters (in particular the hold bonus) were tested.
- The best results were achieved with hold_seconds=0.5 and a pose_error threshold of 0.5.

3.2 Quantitative results

The learning curves for the Humanoid v5 show a steady increase in cumulative reward over the course of training. As shown in Figure 1 (top-left), the episode reward rises from approximately 3000 to over 5000 after 2.4 million timesteps, indicating that the agent is becoming increasingly adept at mimicking the reference motion. The episode length (top-right) increases from approximately 1280 to 1400 steps over the same period, suggesting that the agent is able to maintain stability for longer durations.

The learning rate was held constant at 2e-4 throughout training (bottom-left). The frames per second (FPS) remained stable at approximately 1000-1400 FPS (bottom-right), indicating efficient training throughput.



Training metrics for the Humanoid v5 over 2.4 million timesteps. Top-left: Episode reward; Top-right: Episode length; Bottom-left: Learning rate (constant at 2e-4); Bottom-right: Frames per second (FPS).

A direct quantitative comparison with the v3 model is difficult, as the joint structures differ significantly between the two models. Consequently, the error metrics and reward scales are not directly comparable.

3.3 Qualitative results

From a qualitative perspective, the Humanoid v5 demonstrates a recognisable burpee movement.

The standing phases (stand, squat, jump, land) are performed steadily.

During the ground-based phases (plank, push-up), however, the robot tends to slump. It is unable to hold the position for any length of time, which suggests insufficient balance control during these dynamically unstable phases.

Standing up is also an issue which leads us to the conclusion, that iff we had more time, we should try to create a seperate learning branch for teaching the robot how to stand up again after a push-up etc.. This should dramatically help to get a full cycle.

Currently a training run takes approx. 47-58 min.

[TODO: Embed/link rendered videos of the trained policy, e.g. from videos/ (Google Docs: link out, since video can't embed directly)]

3.4 Reproducibility

Exact steps to regenerate these results:

You can also find this in the github repo under readme.md

How to Setup:

```
git clone https://github.com/SvnNgbr/nca-sn-lw-mujoco
cd nca-sn-lw-mujoco
conda create -n mujoco_3_11_15 python=3.11.15
conda activate mujoco_3_11_15
python setup.py


How to Run:

python run.py

# only V5 training

python run.py --mode train

# only V5 video with existing modell

python run.py --mode video --skip-train

# V3 training + video

python run.py --version v3

# V5 video with root-assist

python run.py --mode video --skip-train --root-assist 0.5
```

3.5 Do the results address the stated problem?

The results go some way towards addressing the problem. The agent is able to perform a burpee-like movement.

However, the system is not yet sufficiently robust. In particular, the ground contact phases are unstable. This is a well-known challenge when it comes to imitating ground contact movements with humanoid robots.

[TODO: Does the agent perform a recognizable, stable burpee end-to-end? Where does it fall short?]

4. Discussion

4.1 Did we solve the problem?

We have not fully resolved the problem. The agent performs a burpee-like movement, but not with the required stability.

However, the work demonstrates that the chosen approach (reference tracking + residual RL) is, in principle, transferable to a new model.

The defined methodology (v5ref.py, v5envtrain.py, etc.) provides a solid basis for further optimisations.

We struggle with the connection of moves required for the burpee (especially the standing up)

4.2 Big / unexpected challenges

The biggest challenge was adapting the joint structure and the angle signs to the Humanoid v5. An interactive tester (v5posmove.py) was essential here.

The hold-reward concept had to be adjusted iteratively. Requirements that were too high led to frustration for the agent, whilst requirements that were too low had no effect.

The different physics of the v5 (mass distribution, joint limits) led to unexpected behaviour, which could only be compensated for after several training runs.

Previous iterations and tests consumed a lot of time, especially starting to work with the V5 without knowing the basics that we later learned through the creation of BurpeeImitationV2.

4.3 What's still missing

There is no explicit balance component that is specifically trained for ground-based phases.

An alternative would be to use a separate stand-up training method, as described in the literature for humanoid robots.

The next step would be to transfer the trained model to a more realistic scenario (e.g. with noise or varying ground conditions) in order to test its robustness.

4.4 Personal learning curve (meta)

The study has shown that the choice of reward function has a decisive influence on learning behaviour.

The importance of interactive debugging tools (such as the Pose Tester) for the development of RL environments has become clear.

Transferring a model to a new environment is not straightforward and requires a deep understanding of the underlying physic differences and the model parameters.

Appendix / Notes

[TODO: Any additional background tasks worth documenting — e.g. environment setup pain points, dataset/model licensing]